

SONNX: Towards an ONNX Profile for critical systems

Eric Jenn, Jean Souyris, Henri Belfy, Loïc Correnson, Mariem Turki, Nicolas Valot, Franck Vedrine

▶ **To cite this version:**

Eric Jenn, Jean Souyris, Henri Belfy, Loïc Correnson, Mariem Turki, et al.. SONNX: Towards an ONNX Profile for critical systems. ERTS 2026 - 13th European Congress of Embedded Real Time Systems, Feb 2026, Toulouse, France. ⟨10.82331/ERTS.2026.2⟩. ⟨hal-05513194⟩

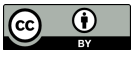
HAL Id: hal-05513194

https://hal.science/hal-05513194v1

Submitted on 16 Feb 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

SONNX: Towards an ONNX Profile for critical systems

Eric Jenn[1], Jean Souyris[2], Henri Belfy[3], Loïc Correnson[4], Mariem Turki[5], Nicolas Valot[6], Franck Védrine[4]

[1,5]*IRT Saint-Exupéry, Toulouse, France*
first.last@irt-saintexupery.com

[2]*Airbus Commercial Toulouse, France*
jean.souyris@airbus.com

[3]Thales AVS, *Toulouse, France*
henri.belfy@fr.thalesgroup.com

[4]*CEA LIST, Saclay, France*
first.last@cea.fr

[6]*Airbus Helicopters, Vitrolles, France*
nicolas.valot@airbus.com

ABSTRACT

This paper presents the objectives and the first results of the SONNX Working Group hosted by the ONNX community. SONNX aims at completing the ONNX standard and providing additional artifacts to support the development and certification of critical systems embedding ML algorithms.

1. INTRODUCTION

The integration of machine learning (ML) into safety-critical systems presents significant challenges for multiple reasons including the fundamental and intrinsic probabilistic nature of AI algorithms, the difficulty to explain the outputs produced and, more globally, the significant changes brought to well-established system and software development practices. A large number of publications already discuss these challenges. For instance, an in-depth discussion about these issues in the context of the development of certified systems can be found in [1].

In this paper, we focus on one specific activity of the development of a system embedding ML: the one that transforms the representation of a trained model into an executable implementation of the very same model. More precisely, we discuss the case where the functional specification of the model is described using the ONNX standard (https://onnx.ai/). The ONNX model may be either directly interpreted by a tool or translated manually or automatically into some lower level, semantically equivalent, representation (e.g., some C source code, some intermediate representation, etc.). In both cases, to be able to interpret or translate the model according to the intent of its designer, its syntax and semantics must be perfectly clear and non-ambiguous.

This requirement is applicable to any system, but it is of course of utmost importance for systems whose failure may have a critical impact on business, safety, etc. In the aeronautical domain, for instance, evidence must be provided to show that the model's semantics are preserved during implementation—something that is only possible if those semantics are precisely defined from the outset.

The current ONNX standard, designed and used for "general purpose AI", does not fully satisfy these requirements. Therefore, there is a need to (i) clarify the industrial requirements in that matter (i.e. *what do we really require of the ONNX model?*), and (ii) identify and address the weaknesses of the current standard in a systematic way, (iii) produce a specific *profile* that would correct issues identified in (ii) to comply with requirements specified in (i), without diverging from the ONNX standard.

In this paper, we describe the objectives, the approach, and the first results obtained by the SONNX Working Group that is currently developing this "Safety-related ONNX profile" (at https://github.com/onnx/working-groups/tree/main/safety-related-profile).

The paper is organized as follows: Section 2 presents related work; Section 3 defines the main objectives targeted by SONNX and the challenges addressed; Section 4 presents the first outputs of the Working Group; and Section 5 concludes the paper.

2. RELATED WORK

To the best of our knowledge, no other initiative is addressing specifically the ONNX standard to improve its documentation and reference implementation to address the concerns of critical systems. In general, standards such as ONNX, TensorFlow, Keras, etc., provide the necessary and sufficient documentation for a user to understand the general purpose of *operators*, i.e., mathematical operations applicable to tensors, such as an addition, a convolution, etc. They sometimes refer to other implementations, such as numpy's, but without providing a systematic and precise definition of the constraints applicable on the inputs (arguments and attributes), the behavior in case of error, the constraints applicable to specific data types, etc. To obtain this information, the user has either to read the source code (when available) or to test the implementation to infer those constraints. However, the NNEF (Neural Network Exchange Format) initiative [2], targets similar objectives as SONNX.

Eric Jenn et al. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0, which permits unrestricted use, distribution, and reproduction in any medium, provided the original author and source are credited.

NNEF is an open standard developed by the Khronos Group to enable efficient cross-platform deployment of neural network models. As ONNX, it provides a well-defined, extensible model serialization format ensuring interoperability between training frameworks and inference engines. NNEF is particularly geared toward embedded and edge devices, emphasizing portability and hardware acceleration. It comes with a very good documentation and its serialization format is human readable whereas ONNX' relies on Google's protobuf serialization mechanism. However, ONNX was embraced early by major ML frameworks like PyTorch and TensorFlow, making it easy to export/import models. The need to interoperability being already satisfied by ONNX, NNEF never got strong backing from those major frameworks and adopted by end-users. Converters exist, but integration is not seamless or well maintained.

Our initiative is tightly related to the emerging industrial standards, in particular in the aeronautics and automotive domains. It is in particular addressing some of the concerns of the AS6983/ED-324[1] [3] currently developed by the joint SAE/EUROCAE WG114 working group. Figure **1** gives an overview of the so-called W-process proposed by the AS6983/ED-324. In this process, the ML Model Description (MLMD) represents the pivot artifact used to start the development of the ML software and hardware items. It captures hyper parameters, parameters, and the computation graph (composed of a set of interconnected operators).

We can also link our work to the objectives given in the publicly available EASA's Concept Paper [4]. For instance, objective LM-15 states that "the applicant should capture the description of the resulting ML model" and objective IMP-02 requires the applicant to "validate the model description captured under Objective LM-15[...]". Both objectives imply the existence of some well-defined formalism to support this description. Those objectives can also be found in [5] that refers to the ARP6983/ED-324 as an acceptable means of compliance (§"Learning verification", item e.8 page 28 and "ML Model Implementation", items f.1 and f.2).

3. Objectives and Challenges

3.1. Overview of ONNX

ONNX (Open Neural Network Exchange) is an open-source format designed to represent machine learning models in a framework-agnostic way. Microsoft and Facebook initiated it to facilitate interoperability between popular ML tools such as PyTorch, TensorFlow, Keras, and scikit-learn. The ONNX format defines a standard set of operators (or "opset") and a computational graph model that allows models trained in one framework to be exported, then imported and run in another. This simplifies model deployment and optimization, especially in heterogeneous environments where different hardware targets such as CPUs, GPUs, FPGAs, or specialized accelerators, are involved. ONNX models are supported by a wide ecosystem of tools and runtimes, such as ONNX Runtime, Xilinx' VITIS AI, ST Microelectronics' STM32Cube.AI, etc.

[1] The document is not yet publicly available. An overview of the expected contents can be found in [3].

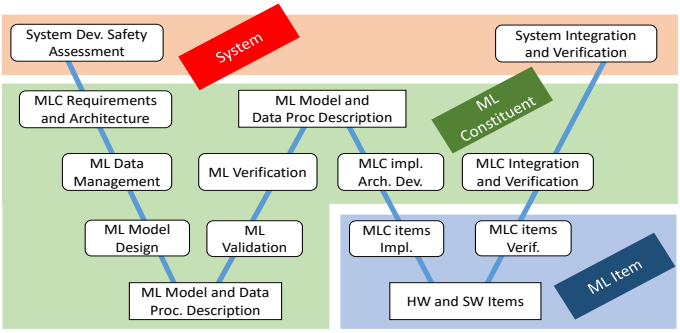


Figure 1. Overview of the ARP6983 process

3.2. Design principles of the SONNX profile

A few principles guiding the construction of the SONNX profile are recalled hereafter. The first one is a condition for the actual adoption of the profile; the others are requirements to get confidence on the implementation of a model.

Compatibility. The profile must be compatible with ONNX, i.e., it shall not change the semantics of operators. However, it can impose some restrictions on the use of these operators, either to facilitate certification or, sometimes, to simplify the development of the profile itself. More details on restrictions are given in Section 4.2.1.

Clarity and readability. The documentation must be understandable by the users and the implementers of the operator, without requiring the prior knowledge of some formal language.

Completeness and precision. The semantics of operators and graph must be completely and precisely described, including the effects of numerical errors (rounding errors, overflows, …) and special numerical values (IEEE754's `Infs` or `NaN`). Note that strict completeness and precision usually requires some mathematical formalization, which is contradictory with the previous principle; therefore, we are providing two levels of specification: an *informal* and a *formal* one.

Explicit control of the operator semantics. SONNX precludes the use of default values for inputs and attributes to (i) enforce a strong traceability between the model and its implementation, and (ii) ensure that the operator behavior is actually under control by its user.

Determinism of behavior and resource usage. This last principle has actually little impact on the SONNX specification for it is generally a matter of implementation choice. For instance, the idea is to preclude or limit the use of constructs that would lead to a non-deterministic usage of resource. It is worth noting that operators generating random number must be deterministic too, i.e., they must generate the same sequence of random numbers for a given seed.

3.3. Issues and challenges

ONNX is largely used in the AI/ML community, and most of the frameworks and tool are able to produce or consume ONNX models. This popularity is due to several technical and non-technical reasons among which being backed by

Microsoft, Meta, Intel AMD, IBM and other key players is certainly not the least. The quality of the documentation and software artifacts is sufficient for most usages – the empirical proof being the large usage of ONNX in the ML community –, and the semantics of the operators is *de facto* defined by the implementations themselves. In particular, the semantics of any operator (e.g., **Conv**) is perfectly defined by ONNX Runtime or by any other conversion and implementation chain. However, the question is to know if these semantics are the same.

A simple test done on ONNX Runtime and the ONNX "ReferenceEvaluator", clearly shows that this is not the case[2]. More generally, the work presented in [6] shows that ONNX converters suffer from numerous failures, some of them being due to a "bad implementation of the standard". This paper gives some of direct cause for these errors, but it does not search for the primary cause, which *may* be rooted in an incomplete specification and the interpretation that was then necessary to implement the operator. In any case, this level of interpretability is clearly not acceptable for a ML model aimed at being used in a critical application.

Issues concerning the ONNX specification – and violating the principles stated earlier – can be organized into several categories:

- insufficient documentation
- use of features raising certification concerns
- choices left to the implementer.

Insufficient documentation. The semantics of operators is sometimes considered so obvious that their description is minimalistic. For instance, the documentation of the convolution operator simply states, "The convolution operator consumes an input tensor and a filter, and computes the output"[3]. The value used for padding (0) is not specified either.

In practice, this is usually not a problem since most developers use ONNX as an exchange format between tools but actually rely on some well-documented API to build their models. For instance, PyTorch, gives an extensive description of the same convolution operator including a mathematical description, the formulae to compute the output tensor shapes, etc. However, this specification is insufficient to develop (and verify) the operator. Several examples are given hereafter.

First, the effects of special values such as the IEEE754's `inf` and `NaNs` on the behavior of operators is either not clearly specified or is sometimes counter intuitive. For instance, while the **Clip** operator use to saturate values is intuitively defined as **Clip**(a,L,M)=**Max**(L,**Min**(a,M)) and considering that **Max**(a,`Nan`)=`NaN` and **Min**(a,`NaN`)=`NaN`, we would expect the operator to return a `NaN` whenever either L or M is `NaN`, but the actual behavior of the operator is that L (resp. M) is replaced by the min (resp. max) value of the type if L (resp. M) equals `NaN`.

Second, the behavior in case of errors is not specified. In particular, nothing is said about the fact that a floating-point (FP) operator may return an `Inf` or a `NaN`, and the conditions in which this may happen. This is usual for FP operations since the behavior of a FP operator usually implicitly refers to the IEEE 754 standard. However, things are very different for integer operators since their behavior may be implementation-dependent (e.g., in the case of an integer division by zero) or not intuitive (e.g., in the case of an overflow with wrap-around).

Certification concerns. Besides the weaknesses of parts of the documentation, other issues concern some features introduced in the standard to simplify the developer's activity but raising certification concerns. For instance, the shape inference capability of ONNX[4] strongly simplifies the work of the model developer, but it also introduces implicit model modifications that make verification and traceability between the model and its implementation harder. In addition, shape inference is not (yet) very well documented and one has to rely on the source code to get a clear understanding of what is done "under the hood" by the framework. The same remark can be done for broadcasting, i.e., the automatic expansion of tensor dimensions so that two tensors of different shapes can be used together in an element-wise operation. In the first case, SONNX simply forbids shape inference and imposes shapes to be explicitly set according to shape compatibility constraints (for instance, if *A* and *B* are two tensors then the shape of tensor *Y* in *Y*=**Add***(A,B)* must be set and be the one of *A* and *B*, possibly after broadcasting).

In the same way, default operator attribute values are also a feature that we would like to avoid in critical systems. For instance, the ONNX convolution operator (**Conv**) applies an implicit zero padding if the pads attribute is not set. In the SONNX profile, we require the user to specify explicitly the padding for each axial axis, which also implies the `auto_pad` attribute to be set to "`NOT_SET`".

Choices left to the implementer. Some elements of the model are deliberately and explicitly left to the interpretation of the implementer. For instance, concerning operator overloading[4], the documentation states that "*The (domain, name, overload) tuple must be unique across the function* prototypes *in this list. In case of any conflicts the behavior […] is defined by the runtimes*". The order in which operators are executed is also left to the implementer as long as it complies with a topological order. This obviously makes sense since the semantics of the model does not depend on the actual order, but it may be cases where the model designer wants a specific order to be followed.

3.4. Scope and Requirements

The SONNX standard currently covers only a subset of the ONNX operator set. The choice of the "baseline" operators has been done by the industrial members of the work group in order to support their use cases: real-time object detection using YOLOV8 and MobileNetSSDv2, video tracking using

[2] The "ReferenceEvaluator" is a pure python implementation of the ONNX specification (see https://onnx.ai/onnx/api/reference.html). Note that the documentation clearly warns the user about the discrepancy that may be observed with the ONNX specification.

[3] https://onnx.ai/onnx/operators/onnx__Conv.html

[4] https://github.com/onnx/onnx/blob/main/onnx/onnx.proto3

VideoSwin, etc. The complete list is available in the working group's repository[5].

4. The SONNX Working Group and the SONNX Profile

4.1. The SONNX Working group

The SONNX Working (WG) is one of the working groups hosted by the ONNX community. It was established in September 2024 and has met every two weeks since then. Participants come from academia (ONERA, INRIA, CEA, IRT, etc.) and industry (Airbus, Thales, Bosch, Critical Software, Collins, etc.). We expect the final SONNX profile (described in the next section) to be domain-agnostic, but, as of today, there is a strong bias towards aeronautical domain and its regulation. This is justified by the very strong representation of industrialists in this domain in the working group, which include representatives of Airbus Commercial, Airbus Helicopter, Thales AVS, etc. This focus is also justified by the relative maturity of the regulatory recommendation standards in this domain (see Section on related work).

Our initial timeline was to provide a series of industry-specific requirement specifications and scope definition that would be the basis for the development of the SONNX scope and specification guidelines by the end of 2024, and deliver a first version of the profile by the end of 2025. In reality, we expect a first complete profile (covering a subset of the ONNX operators) to be delivered by the end of 2026. In any case, all the elements produced by the working group (specification, analyses, meeting minutes, etc.) are available on the ONNX repository[5].

4.2. The SONNX Profile

In ONNX, a Machine Learning model is represented by an acyclic directed graph where a node represents an instance of some *operators* applied on tensors (e.g., **Add**, **Mul**, **Conv**, etc.), and an *edge* represents an oriented flows between the output of one node and the input of another. Executing a graph means loading it from its serialized representation (i.e., a file), and executing all nodes that have their inputs available until no more nodes can be executed[6]. The fact that the graph is acyclic guarantees that this process terminates.

Accordingly, the SONNX profile is composed of three main elements: (i) the specification of the operators included in the profile, (ii) the specification of the graph execution semantics, and (iii) a file serialization format. The file format is, of course, an important component as it is the concrete language used to describe a graph. However, since it is already well defined through Google's protobuf, we expect it to pose few issues and have therefore planned to address it at a later stage. Consequently, our efforts have focused primarily on the first two components.

4.2.1. The Operators

For each operator, the SONNX profile provides three main components: an *informal specification* (or "documentation"), a *formal specification*, and a *reference implementation*.

Figure 2 illustrates how these elements relate: beginning with the informal specification ❶, a first level of abstract formalization is produced ❷, which is then refined ❸ to include concrete constructs such as arrays and loops suitable for automatic translation into C. The C implementation of an operator ❹ (called SONNX reference implementation) is not aimed at being embedded, but it can serve as an oracle to test the end-user's optimized version of the implementation ❺. Those elements are detailed hereafter.

Informal specification. This component describes the behavior of the operator as completely and "intuitively" as possible. This specification is essentially aimed at the user of the operator, i.e., the one that designs the ML model. As in the original ONNX documentation, the inputs, outputs and attributes are specified, but emphasis is placed on (i) the possible restrictions introduced by the SONNX profile and (ii) the constraints to be satisfied for the operator to make sense. Figure 3 gives an excerpt of this documentation for the **Conv** operator. Each operator is specified for real numbers ($\mathbb{R}$) and each of the applicable implementation data types, e.g., floating-point numbers (float32, float64, etc.), integer numbers (int8, uint8, int16, uint16, etc.).

Restrictions are imposed either to satisfy certification-driven industrial requirements (such as forbidding default values) or to temporarily simplify the specification process, provided they remain consistent with current industrial use cases. For instance, as shown on Figure 4, SONNX restricts the input tensor of the **Conv** operator to have at most two spatial dimensions (restriction on shape) and convolution to be either

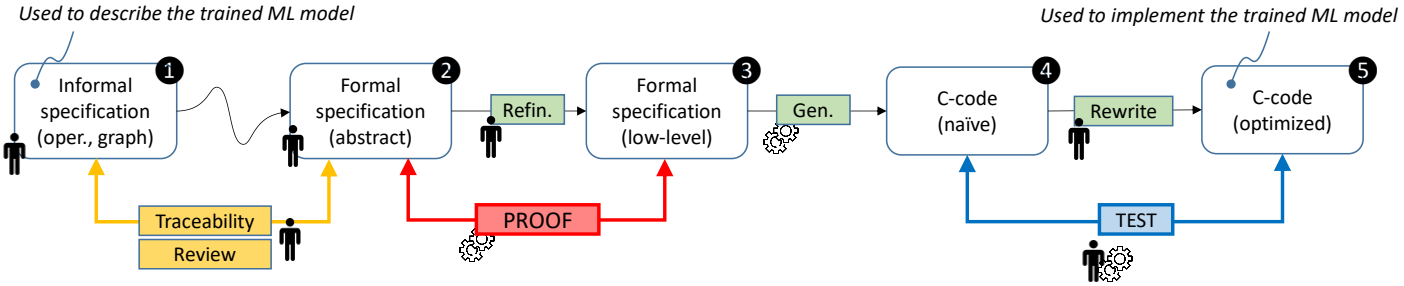
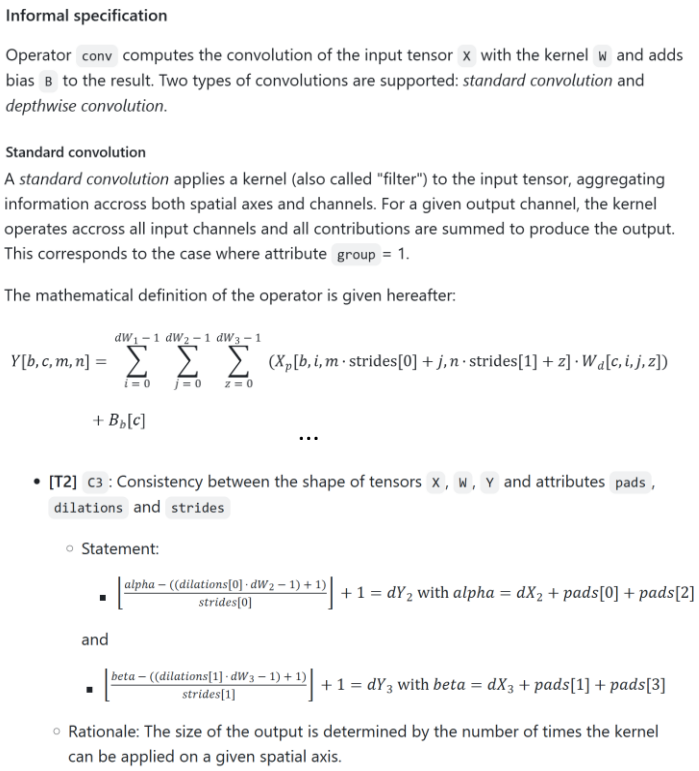


Figure 2. From formal specification to implementation

[5] The latest developments are available at https://github.com/ericjenn/working-groups/blob/ericjenn-srpwg-wg1/safety-related-profile/deliverables/scope/scope.md

[6] In reality, things are slightly more complicated since we have to consider control flow nodes such as **If**, **Loop**, and **Scan**, but those operators are not considered in the first version of the profile.



Informal specification

Operator `conv` computes the convolution of the input tensor `X` with the kernel `W` and adds bias `B` to the result. Two types of convolutions are supported: *standard convolution* and *depthwise convolution*.

Standard convolution

A *standard convolution* applies a kernel (also called "filter") to the input tensor, aggregating information accross both spatial axes and channels. For a given output channel, the kernel operates accross all input channels and all contributions are summed to produce the output. This corresponds to the case where attribute `group` = 1.

The mathematical definition of the operator is given hereafter:

$$Y[b,c,m,n] = \sum_{i=0}^{dW_1-1} \sum_{j=0}^{dW_2-1} \sum_{z=0}^{dW_3-1} (X_p[b,i,m\cdot \text{strides}[0]+j, n\cdot \text{strides}[1]+z]\cdot W_d[c,i,j,z]) + B_b[c]$$

...

- [T2] `C3` : Consistency between the shape of tensors `X`, `W`, `Y` and attributes `pads`, `dilations` and `strides`
  - Statement:
    - $\left\lfloor \frac{alpha-((dilations[0]\cdot dW_2-1)+1)}{strides[0]} \right\rfloor + 1 = dY_2$ with $alpha = dX_2 + pads[0] + pads[2]$

    and

    - $\left\lfloor \frac{beta-((dilations[1]\cdot dW_3-1)+1)}{strides[1]} \right\rfloor + 1 = dY_3$ with $beta = dX_3 + pads[1] + pads[3]$
  - Rationale: The size of the output is determined by the number of times the kernel can be applied on a given spatial axis.

Figure 3. **Conv** operator informal specification (excerpt)

standard or depthwise convolution (restriction on attributes). These two restrictions are aimed at facilitating the formal specification and reducing the verification effort. They may be relaxed in a later version of the profile depending on the end users' needs.

Then, the operator is specified using (i) natural language, (ii) some mathematical formula when deemed necessary and (iii) examples. The objective is to make the semantics of the operator understandable by both the user and the implementer of the operator, without requiring them to master a formal language.

In addition, elements of the informal specification are tagged in order to facilitate the traceability with the formal specification and other verification artifacts (e.g., tag **[T2]** in Figure 3).
The informal specification also clearly expresses the constraints applicable to the inputs, outputs and attributes of the operator.
Those constraints define the domain in which the specification of the operator makes sense. For instance, Figure 3 shows a constraint (C3) that relates the shapes of the input tensor X, the kernel W, the output tensor Y and the padding and dilation attribute (notation dT_i represents the size of the i -th dimension of tensor T).

Formal specification. The formal specification is expressed using WhyML, the formal language of the Why3[7] deductive program verification platform [7]. WhyML is a strongly typed functional language extended with imperative features that supports algebraic data types, recursive functions, pattern matching, mutable variables, etc. It is used to write programs or formal specifications, annotate them with properties such as invariants or pre/post-conditions. The Why3 environment

Rest.	Statement	Origin
R1	Input tensor X has 2 spatial axes	Transient
R2	Attribute auto_pad is set to NOTSET	No default values
R3	Attribute group is set to 1 (standard convolution) or to the number of channels of the input tensor X (depthwise convolution)	Transient

Figure 4. Restrictions for the **Conv** operator

provides a verification condition generator (VCG) that transforms annotated code into logical proof obligations, and a prover interface that connects to automated and interactive provers (e.g., Alt-Ergo, Z3, CVC4, CVC5, Rocq, Isabelle).

The formal specification serves several purposes. First, it provides a rigorous and mathematically sound specification of the operators that complements the informal specification. It is complementary in the sense that it specifies completely the operator whereas the documentation remains informal and usually incomplete in order to remains easily readable and understandable.

In the following paragraphs, we briefly present the formal specification of the convolution operator.

Figure 5 gives an excerpt of the formal specification of the **Conv** operator, at two levels. At the most abstract level, the specification essentially uses recursive definitions that simplify the proof process. At the concrete level, the specification uses constructs, such as loops, that makes it very close to the actual implementation in C. The link between the two specification levels is expressed using the "ensures" clause indicated in red. Formal proof is used to establish the strict equivalence of the two models or, stated differently, the correctness of the refinement.

The formal specification of **Conv** relies on the formal specification of tensors, which is depicted in Figure 6. In order to support the refinement down to implementation, tensors are represented in two different ways. First is the abstract Tensor, which is modeled as a tuple composed of a list of integers representing the size of each dimension (the shape of the tensor), a function mapping some index to a value, and a "background" value representing the default value of the function, to make it completely defined on its domain. Second is the more concrete representation of tensors, called `Ctensor`, defined by its rank (`t_rank` giving the number of dimensions), and an array of integers (`t_dims`) specifying the size of each dimension. It also contains a pointer (`t_data`) to a 1D array in which the actual data are stored. `iarray` and `farray` represent pointers to block of memory containing integer and floating-point values, respectively. This representation is close to how a tensor would actually be naturally implemented in C, our target language.

[7] https://www.why3.org/

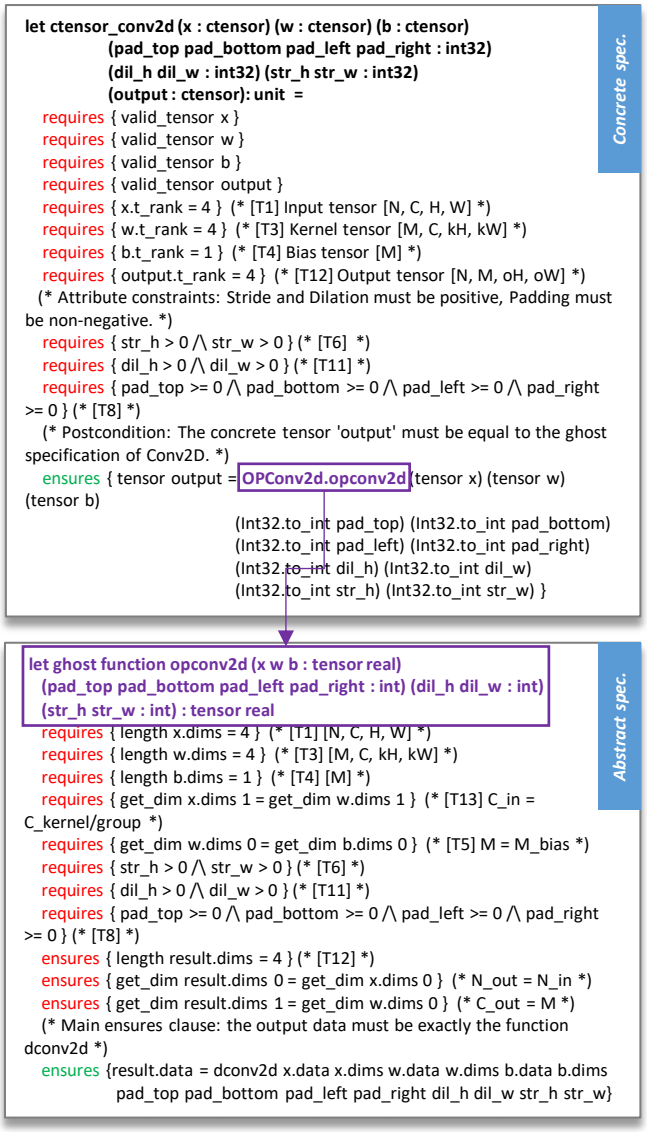


Figure 5. **Conv** operator formal specification (excerpt)

The "ghost" function "`tensor`" associates the abstract and concrete representation of tensors (see Figure 6). It takes a concrete `Ctensor` and converts it into its equivalent pure function abstract tensor definition. In essence, this function acts as the formal bridge in the proof process, asserting that the collection of low-level data stored in the C structure is mathematically identical to the abstract tensor required by the formal specification.

It is worth noting that the non-purely logical part of the WhyML formal specification is executable, which means that testing can be used to validate the specification. Additionally, validation can also be carried out by introducing and proving goals expressing specific expected properties of the specification.

Numerical accuracy. Numerical accuracy raised numerous discussions in the working group, the main question being: *shall SONNX specify the numerical accuracy of operators?* An example of such a requirements could be "for any input, the numerical error on the output of operator **op** must be less than $10^{-10}\times A$, where A the maximal absolute value of the tensor coefficients".

Indeed, numerical accuracy represents an important question for at least three reasons. First, the result produced by a graph

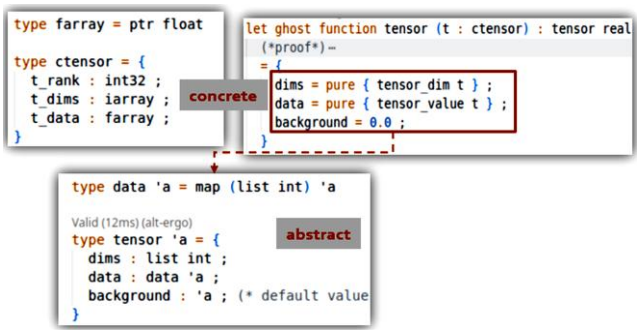


Figure 6. Abstract and concrete model of a tensor (excerpt)

obviously depends on the results computed at the operator level and, due to the non-linearity of the graph, a decision boundary may be crossed for a very small perturbation of the computation, leading to an erroneous result. In addition, the number of operations realized in a neural network (esp. deep ones) is extremely large, making very difficult to assess the impact of an error in an operator on the final output or, reciprocally, to determine the necessary accuracy at operator level to get a correct output. Moreover, requirements about operator accuracy are rarely defined *a priori* even in more conventional cases, except in some very specific scenarios. If necessary, accuracy is assessed at function or system level.

Second, computation errors may be different from one implementation to another. Variability may be due to several reasons including optimizations done by the implementation framework (e.g. merging a batch normalization operator with a linear operator), choice of the algorithm used to implement the operator (e.g., "naive" method, im2col and GEMM, Winograd, FFT for a convolution), use of non-IEEE 754-compliant hardware, etc. This variability may lead to different behaviors when executing the same model on different machines or executing the same model several times on the same machine.

Third, problems with numerical accuracy may lead to the invalidation of some of the credits sought at higher level on the model. For instance, proof of robustness done on the model using formal methods (e.g.,Marabou 2.0 [8]) may not be preserved due to computation errors.

In practice, note that neural networks tend to be "naturally" robust to numerical errors. This is in particular due to the fact that networks are trained on real data with a much larger noise than rounding, the use of optimizations such as drop-out that "desensitize" network to small perturbations, the use of continuous activation functions (e.g., ReLu), the use of various normalizations (e.g., batch normalization) that enforce stability.

In SONNX, we have adopted the following pragmatic approach concerning numerical errors: rather than specifying a value for the error bound (e.g., 10^{-10}), we propose:

- an analytical expression of an upper bound of the error for each operator considering a naive implementation of

the operator, namely, the one provided as part of the reference implementation (see Section 4.3).
- a technical means to estimate the error of a C/C++ implementation, by using the C++ operator overloading mechanism.

A simple example of this approach is given hereafter in the case of the **Div** operator. Figure 7 expresses how errors on the inputs propagates to the output and how the operation itself introduces some new error.

Figure 8 shows the code that can be used to (i) estimate in a conservative upper bound on the error on the result (`c_err`) and (ii) check that is it less than the is less or equal to the cumulated error. An operator implementer can use this code, which relies on the `fldlib` library [9], to estimate the error introduced by his/her implementation, and check compliance with the specified error bound.

Numerical Accuracy

For $Y = A \ / \ B$ with numerical errors A_{err}, B_{err}:

Error Propagation

For all valid indexes i where $B[i] \neq 0$ and $B[i] + B_{\text{err}}[i]$ does not cross zero,

$$|Y_{\text{err}}^{\text{propag}}[i]| \leq \left|\frac{A_{\text{err}}[i]}{B[i]}\right| + \left|\frac{A[i] \cdot B_{\text{err}}[i]}{B[i]^2}\right|$$

Error Introduction

Floating-point division introduces rounding error bounded by the unit in the last place (ulp) of $Y[i]$ in the target floating-point format.

- $Y_{\text{err}}^{\text{intro}} = ulp(Y[i])$.

Figure 7. Numerical accuracy specification for the **Div** operator (excerpt)

```
Tensor<SymbolicDomainError> A, B;
auto result = [&A,&B](auto i) {
  if (B[i].real != 0) return A[i] / B[i];
  if (A[i].real != 0) return
SymbolicDomainError::inf();
  return SymbolicDomainError::nan();
};
for (auto i : A.indexes()) {
   auto a = A[i];
   auto b = B[i];
   auto c = result(i);
   if (std::isfinite(b.real) && b.real != 0 && b.real +
b.err != 0) {
      double bound = std::abs(a.err / b.real) +
          std::abs(a.real * b.err / (b.real * b.real));
      assert(std::abs(c.err) <= bound + ulp(c.real));
   }
   if (b.real == 0 && a.real != 0){ assert(c.is_inf());
}
   if (b.real == 0 && a.real == 0){ assert(c.is_nan());
} }
```

Figure 8. Computing error bounds for the **Div** operator

4.2.2. The graph model

A ML model is a computation graph where nodes represent operators and edges represent the data paths between them. The function performed by the model depends on the semantics of each operator (see previous section), on the sequencing of their execution, and on the assignments of their inputs and outputs.

In ONNX, the semantics of the graph is described in the "Concept" section of the on-line documentation[8]. In SONNX, we develop a specific informal and formal specification for the graph. Preliminary versions are given in Figure 9 and 10. At the time of writing, this work is still in progress. In particular, the particular case of the control flow operators (**if**, **loop**, **scan**) remains to be addressed, but should eventually comply with the very general principles stated in Figure 9.

Graph

- `[T01a]` A graph is an acyclic graph composed of *nodes* and *edges*
- `[T01b]` A node is either a *tensor node* or a *computation node*

Execution Semantics

Executing a graph means evaluating the output tensors of the graph according to the following rules:

- `[T06a]` A computation node is executable if all the tensors connected to its inputs (i.e., belonging to an edge) are initialized
- `[T06b]` Executing a computation node means assigning values to the tensors connected to its outputs (i.e., belonging to an edge) so that the relation specified by the operator between its inputs and outputs holds
- `[T06c]` All executable computation nodes shall be executed
- `[T06e]` A tensor shall be assigned a value at most once (Single Assignment)

Figure 9. Informal specification of the graph execution semantics (excerpt)

```
(*   ------------------------------------------------------------
*)
(* Execute one node                                            *)
(*   ------------------------------------------------------------
*)
let ghost exec_node (s: graph_state) (n: node) : graph_state
(* [T05a] The node must be ready to be executed *)
requires { node_is_ready s n }
(* [T05a] Inputs # must match the number of operator's inputs *)
requires { length n.oi = length n.ope.opi }
(* [T03b] Output # must match the number of operator's outputs
*)
requires { length n.ou = length n.ope.opo }
(* After execution, all output tensors shall be set *)
ensures      {   forall   t:   tensor_id.   mem   t   n.ou   ->
tensor_is_initialized result t }
=
    (* the values of tensors that are inputs to a node *)
    let inputs = map (fun t -> fget s t) n.oi in
        assert { forall v.  mem v inputs -> v <> None  };
    (* the values of all outputs after evaluation *)
    let outputs = eval_operator n.ope inputs in
        assert { forall v.  mem v outputs -> v <> None  };
        (* the updated state *)
        assign_list s (zip n.ou outputs)
```

Figure 10. Formal specification of the graph execution semantics (excerpt)

[8] https://onnx.ai/onnx/intro/concepts.html

4.3. The reference implementation

For each operator, we provide a simple and traceable reference C implementation generated from the formal specification. This "reference implementation" can be used to verify the correctness of the optimized implementations that will be used in operation, using a "back-to-back" test configuration (see Figure 2). Note that an implementation should preferably be *proved* correct with respect to our formal specification. However, in practice, is very unlikely that it will be the case, in particular because some implementation choices done to accelerate inference (e.g., use a GPU…) will currently be hardly amenable to formal verification. In addition, formal verification remains a complex and costly activity generally restricted to the most critical software, which is not (yet) targeted by AI.

To support this activity, the SONNX Working Group develops sets of test vectors for each operator using the Hypothesis test generation framework[9].

Tests are also used very early during the specification process. Indeed, when writing the SONNX informal specification, we use tests to check our understanding of the specification with respect to the ONNX Runtime implementation on simple examples created manually (e.g., those given as illustrations in the informal specification). This approach may initially seem unconventional, since it would mean that we are developing our informal (and, later, our formal) specification against an implementation. However, this check is actually an additional means to catch errors that would have been missed by reviews. In practice, this has been extremely useful and has led to the discovery of several errors in the specification.

From the formal specification to the reference implementation

The reference implementation is generated automatically in order to ensure *as much as possible* its compliance with the low-level formal specification. Ideally, this process would be carried out by a qualified tool, i.e., a tool providing all development artifacts required by certification standards for software tools (e.g., DO-330/ED-215 [10]). This is not yet the case for the Why3 code generator, hence some manual activities are still required to show that it correctly implements the formal specification. Nevertheless, the generated code remains very close and, consequently, traceable to the low-level specification.

Figure 11 gives an extract of code generated for the **Conv** operator. In this listing, one can easily recognize the nest of loops that where already used in the low-level specification.
C code generation from Why3 is tuned by drivers, which defines the mapping between Why3 constructs and their equivalents in C, including type correspondences, function signatures and required library imports. Figure 12 shows an excerpt from the driver we used for the C code generation of the tensor structure and **Conv** operator.

It is worth noting that, (i) to make the informal specification as readable and understandable as possible, (ii) to facilitate the manual verification of the equivalence between the informal specification and the top-level specification, (iii) to facilitate the proof of equivalence between the high- and low-level

```
module std.Cfloat
  syntax type float    "double"
  syntax val zero      "0.0"
  syntax val one       "1.0"
  syntax val f32       "(double) %1"
  syntax val f64       "(double) %1"
  syntax val      ( .+ )     "%1 + %2"   prec 8 8 7
  syntax val      ( .- )     "%1 - %2"   prec 8 8 7
  syntax val      (.-_)      "- %1"      prec 5 4
  syntax val      ( .* )     "%1 * %2"   prec 7 7 6
  syntax val      ( ./ )     "%1 / %2"   prec 7 7 6
  syntax val      ( .= )     "%1 = %2"   prec 11 11 10
  syntax val      ( .< )     "%1 < %2"   prec 11 11 10
  syntax val      ( .<= )    "%1 <= %2"  prec 11 11 10
end
[…]
module libvector.CIndex
  interface "#include <stdlib.h>"
  interface "#include <stdint.h>"
end

module libtensor.CTensor
  interface "#include <stdlib.h>"
  interface "#include <stdint.h>"
  interface "#include \"cindex.h\""
```

Figure 12. Driver from Why3 to C (excerpt)

```
void ctensor_conv2d(struct ctensor x, struct ctensor
w, struct ctensor b, int32_t pad_top, int32_t
pad_bottom, int32_t pad_left, int32_t pad_right,
int32_t dil_h, int32_t dil_w, int32_t str_h, int32_t
str_w, struct ctensor output) {
…
if (x_coords && (w_coords && (b_coords &&
output_coords))) {
 o = n_batches - 1;
  if (0 <= o) {
   for (n = 0; ; ++n) {
    o1 = m_out - 1;
     if (0 <= o1) {
      for (m = 0; ; ++m) {
       o2 = h_out - 1;
        if (0 <= o2) {
         for (oh = 0; ; ++oh) {
          o3 = w_out - 1;
           if (0 <= o3) {
            for (ow = 0; ; ++ow) {
             conv_sum = ((double) 0.0);
              o4 = c_in - 1;
                if (0 <= o4) {
                 for (c = 0; ; ++c) {
                  o5 = kh - 1;
                   if (0 <= o5) {
                    for (k_h = 0; ; ++k_h) {
                     o6 = kw - 1;
                      if (0 <= o6) {
                       for (k_w = 0; ; ++k_w) {
            ih = oh * str_h_i + k_h * dil_h_i;
            iw = ow * str_w_i + k_w * dil_w_i;
            padded_ih = ih - pad_top_i;
            padded_iw = iw - pad_left_i;
                        ...
```

Figure 11. Reference implementation of the **Conv** operator (excerpt)

formal specification, operators are described in the most naive and intuitive way.

For instance, our reference implementation of the convolution uses a naive nested loops algorithm, even though most real

[9] https://hypothesis.readthedocs.io/

implementations will rely on one of the optimized algorithms mentioned earlier.

Towards a complete reference implementation.
Up to now, we have only considered providing a reference implementation of the operators and the graph execution semantics, in order to facilitate the verification of an implementation of the SONNX profile and the unitary level. In reality, a model is an integrated set of operators combined to form a particular computation graph. Therefore, to test a complete graph, we need an infrastructure to read an ONNX model compliant with the SONNX profile and to execute the graph operators in the appropriate order.

To do this, we are integrating the code of the operators in the **Aidge** open source and collaborative deep learning framework developed as part of the DeepGreen and NeuroKit2E projects[10]. This framework, hosted by the Eclipse Foundation, integrates all capabilities to develop models, and train, optimize (quantization, compression,…) deploy and test/evaluate them on a variety of hardware targets. **Aidge** already provides optimized code generators for several hardware targets including MCUs, CPUs, GPUs andNPUs. Work is also on going to integrate the ACETONE avionics code generator [6]. Our plan is to (i) integrate the reference implementation of the SONNX operators in a new back-end, and (ii) verify the compliance of the **Aidge** graph sequencer with the one generated from the formally specified SONNX version. Once completed, this solution will provide an integrated environment with the capability to design a model (i.e., create the model description), develop an efficient, target optimized implementation of the model, validate it against the naïve – yet formal – reference implementation.

5. Conclusion

The SONNX Working Group has established a structured approach for developing an ONNX profile that contributes to the body of evidence expected by forthcoming regulatory standards. By starting from an informal specification of ONNX operators and graph execution semantics that clarifies and strengthens the existing documentation, the group derives an abstract formal specification that is progressively refined to a level amenable to automatic translation into C code. The resulting reference implementation enables systematic back-to-back validation of optimized implementations. In addition, hypothesis-based test scripts are provided to support this verification process and to facilitate future extensions of the profile.

This work, which is still in progress, raises several legitimate remarks.

The first one concerns the pertinence and usefulness of applying a relatively complex formal specification process – usually confined to the most critical applications – for inherently “uncertain” ML algorithms. However, even if the learning curve to handle formal methods is steep and the effort to formally specify and verify the first set of operators has been fairly high, the effort is done once, is highly recurrent (the specifications of operators **Add**, **Mul**, **Div**, etc. are similar), and could easily be shared in the ever-enlarging community of machine learning. The authors hope that experts more familiar with formal specification and verification will continue our work; this is actually necessary as the complexity of operators increases.

The second remark concerns the maintenance of the SONNX profile. Indeed, ONNX is evolving continuously by integrating new operators (e.g., operator **Attention** was added in opset 23), or extending the scope of existing operator by extending the supported data types (e.g., the basic **Add** operator introduced was updated in opset 14 to support (u)int8, (u)int16 arguments). Faced to this situation, maintaining SONNX aside ONNX represents undoubtedly a recurrent effort, but (i) SONNX targets a relatively stable and conservative domain of high integrity systems, (ii) part of the SONNX artefacts will not change significantly when an operator changes (e.g. the informal specification will basically remain the same), (iii) the reference implementation is sufficiently simple (“naïve”) to facilitate the integration of such change with a minimal effort.

As of December 2025, the Working Group has met around 20 times. While the number of operators that are fully specified, verified, and implemented remains limited, the group has made steady progress and several key challenges such as the treatment of numerical accuracy, the handling of error cases, etc., have now been clarified, providing a solid foundation for further advances. However, to maintain and accelerate this progress, additional contributors are needed. With this expanded effort, the Working Group is well positioned to make the SONNX profile an integral part of the ONNX standard.

Future work first targets completing the work for the operators of the SONNX profile not yet covered (specifying, proving, generating code, generating tests, *etc*.) and pursuing the integration of the SONNX operators in a new **Aidge** code generation backend. Developing a SONNX profile compliance checker would also be an interesting by-product of our activity.

Acknowledgment

The work described in this paper is the results of the collaborative of the SONNX working group. We would like to thank all participants to the working group, and in particular Jean-Loup Farges (ONERA), Sebastian Boblest (Bosch), and Jean-Baptiste Rouffet (Airbus Protect), João Machado and Ricardo Moreira da Silva (CriticalSoftware). The authors thank the ONNX Community for hosting the SONNX working group and the DeepGreen project (ANR-23-DEGR-0001) for contributing to funding this activity.

References

[1] F. Mamalet *et al.*, ‘White Paper - Machine Learning in Certified Systems’, IRT Saint Exupéry, Mar. 2021. Accessed: Nov. 28, 2025. [Online]. Available: https://hal.archives-ouvertes.fr/hal-03176080

[2] The Khronos Group, ‘Neural Network Exchange Format (NNEF)’. Accessed: Nov. 28, 2025. [Online]. Available: https://www.khronos.org/nnef/

[10] See https://eclipse.dev/aidge/ and https://deepgreen.ai/en/platform.

[3] C. Gabreau, B. Pesquet-Popescu, F. Kaakai, and Lefevre, Baptiste, 'AI for Future Skies: On-going standardization activities to build the next certifica- tion/approval framework for airborne and ground aeronautical products', in *CEUR Workshop Proc.*, Aug. 2021. Accessed: Nov. 28, 2025. [Online]. Available: https://ceur-ws.org/Vol-2916/paper_10.pdf

[4] EASA, 'EASA Concept Paper : Guidance for Level 1 and 2 machine learning applications, issue 02', Mar. 2024. Accessed: Nov. 28, 2025. [Online]. Available: https://www.easa.europa.eu/en/downloads/139504/en

[5] EASA, 'Proposed detailed specifications and associated acceptable means of compliance and guidance material for AI trustworthiness (DS.AI)', NPA 2025-07 (B). Accessed: Nov. 28, 2025. [Online]. Available: https://www.easa.europa.eu/en/document-library/notices-of-proposed-amendment/npa-2025-07

[6] P. Jajal *et al.*, 'Interoperability in Deep Learning: A User Survey and Failure Analysis of ONNX Model Converters', in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, Vienna Austria: ACM, Sept. 2024, pp. 1466–1478. doi: 10.1145/3650212.3680374.

[7] J.-C. Filliâtre and A. Paskevich, 'Why3 — Where Programs Meet Provers', in *Programming Languages and Systems*, vol. 7792, M. Felleisen and P. Gardner, Eds, in Lecture Notes in Computer Science, vol. 7792. , Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 125–128. doi: 10.1007/978-3-642-37036-6_8.

[8] H. Wu *et al.*, 'Marabou 2.0: A Versatile Formal Analyzer of Neural Networks', Jan. 25, 2024, *arXiv*: arXiv:2401.14461. doi: 10.48550/arXiv.2401.14461.

[9] F. Védrine, M. Jacquemin, N. Kosmatov, and J. Signoles, 'Runtime Abstract Interpretation for Numerical Accuracy and Robustness', in *Verification, Model Checking, and Abstract Interpretation*, vol. 12597, F. Henglein, S. Shoham, and Y. Vizel, Eds, in Lecture Notes in Computer Science, vol. 12597. , Cham: Springer International Publishing, 2021, pp. 243–266. doi: 10.1007/978-3-030-67067-2_12.

[10] 'Software Tools Qualification Considerations', RTCA, Inc., DO-330/ED-215, Dec. 2011.